Docker 网络实现 · 原理

第 7 讲

目标

让容器的"各自独立"里能互相找得到

- 本质: 网络隔离来自 **Network 命名空间**
- 四种模式: `bridge / host / none / container`
- `docker0` 桥 + `veth` 对 + iptables 的组成
- 创建网络 + **容器名即 DNS**

一、为什么需要网络

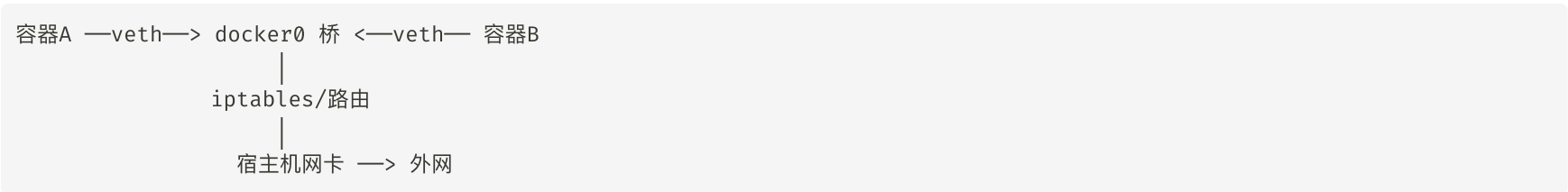
> 容器既独立, 又要互通

上一讲"容器像小机器", 靠 namespace 隔离

- **Network 命名空间** → 每个容器有了"自己的"网卡 / IP / 端口
- `hostname` 是容器 ID · `eth0` 是自己的网卡 · 端口只属于自己
- 但完全隔离 → 组不成"一套应用"

> 既要各管各的, 又要按需牵手, 这就是网络

Docker 网络 = 虚拟网卡 + 网桥 + 一张 NAT 规则



二、四种网络模式

一选模式, 二定结构

容器生下来先选一个网络模式

模式	定性	与宿主机
bridge 默认	同处一个虚拟局域网	NAT 出口 + `-p` 映射
host	直接复用宿主机网络	同 IP、同端口表
none	没有网络	只有 loopback
container	复用另一个容器的	同 IP、同端口表

一行命令看现在有什么网

```
docker network ls
# bridge    host    none    （三个内建）+ 你手动建的
```



三、默认 bridge 的结构

> 一台"交换机"加几根"网线"

docker0 是交换机, veth 是网线

```
   docker0 (172.17.0.1)    ← 桥/交换机
      │       │
   veth0    veth1           ← 虚拟网线
      │       │
   容器A     容器B
172.17.0.2 172.17.0.3
```

- 同桥容器**默认互通** (ping 容器 IP 能通)
- IP 由 Docker 自动分配

出口外网靠 NAT, 入口靠 -p 映射

```
# 出: 容器 → iptables NAT → 宿主机IP → 外网
sysctl net.ipv4.ip_forward      # 置 1 才能出去

# 入: 宿主机8080 → 容器80
docker run -d -p 8080:80 nginx
```



> 宿主机**不能直接**访问容器的 IP/端口, 靠 -p 桥接

四、创建自定义网络

默认 bridge 只认 IP, 不认名字

默认 bridge 的两个"别扭"

- IP 会变: 容器重启后可能换 IP
- 不认名字: 容器**不能**用容器名互访

> 用它互联, 地址动不动就断片

docker network create 起一个"带 DNS 的局域网"

```
docker network create -d bridge mynet
docker network create --subnet 172.20.0.0/16 mynet
docker network create --gateway 172.20.0.1 mynet
docker network create --ip-range 172.20.5.0/24 mynet
```

参数	作用
`-d/--driver`	驱动(默认 bridge)
`--subnet`	指定网段
`--gateway`	网关(桥地址)
`--ip-range`	容器分配IP池

接进同一自定义网 → 容器名即主机名

```
docker run --name web --network mynet nginx
docker run --name app --network mynet python-app

docker exec app ping web            # 直接用名字!
curl http://web                     # web 当域名用
```



> 内嵌 DNS: 把容器名解析成 IP, 重启也不慌

还能指定 IP、加别名、运行中断开

```
docker run -d --name db --network mynet \
  --ip 172.20.5.10                       # 固定 IP
  --network-alias database               # 加别名

docker network connect mynet web        # 运行中接入
docker network disconnect mynet web     # 运行中脱离
```

五、实战一

看透默认 bridge

同桥互通 + NAT + 端口映射

同桥互通: 一个 ping 见真章

```
docker run -d --name mynginx nginx
docker exec -it mynginx ip addr        # 看 eth0 = 172.17.x.x
docker network inspect bridge \
  | grep -A8 Containers                # 桥视角看谁接在网里

docker run --rm -it alpine sh
apk add iputils && ping 172.17.0.x     # 同一 docker0, 通
```



外部访问: -p 把网关到容器

```
docker run -d --name web -p 8080:80 nginx
curl http://localhost:8080    # 宿主机8080 → 容器80
```



> NAT 出、-p 入, 一进一出两个方向

六、实战二

创建并使用自定义网络

容器名当域名用

一手建网, 一手把容器接进去

```
docker network create --driver bridge \
  --subnet 172.20.0.0/16 --gateway 172.20.0.1 \
  --ip-range 172.20.5.0/24 devnet

docker run -d --name web --network devnet nginx
docker run -d --name db  --network devnet mysql:8
docker network inspect devnet | grep -A6 Containers
```



容器名即主机名, 直接互访

```
docker exec -it db nslookup web        # 名字被解析
docker exec -it db curl http://web     # 用名字访问 nginx

docker run -d --name app --network devnet \
  --ip 172.20.5.20 --network-alias backend python:alpine
docker exec -it web sh -c "getent hosts backend"  # 别名也行
```



一张表带走全部要点

方面	要点
本质	网络隔离来自 **Network 命名空间**
模式	bridge(默认)/ host / none / container
bridge	docker0 + veth + iptables; 同桥互通 · NAT 出 · -p 入
创建网络	create -d --subnet --gateway --ip-range
核心价值	自定义网络内嵌 DNS → **容器名即主机名**
细节	–ip 固定 · connect/disconnect · --network-alias

默认网只认 IP, 交了学费才懂自定义网香

> 下一步: 外部访问容器、容器互联(-link)、配置 DNS —— 全部基于本讲的桥与自定义网络

谢谢大家

Q & A